# Concolic Testing for Deep Neural Networks

Sun, Y., Wu, M., Ruan, W., Huang, X., Kwiatkowska, M., & Kroening, D. (2018). Concolic Testing for Deep Neural Networks. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering* (pp. 109-119). Association for Computing Machinery. https://doi.org/10.1145/3238147.3238172

**Published in:**
Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering

**Document Version:**
Peer reviewed version

**Queen's University Belfast - Research Portal:**
Link to publication record in Queen's University Belfast Research Portal

# Concolic Testing for Deep Neural Networks

Youcheng Sun[1], Min Wu[1], Wenjie Ruan[1], Xiaowei Huang[2], Marta Kwiatkowska[1], and Daniel Kroening[1]

[1]University of Oxford, UK
{youcheng.sun; min.wu; wenjie.ruan}@cs.ox.ac.uk
{marta.kwiatkowska; daniel.kroening}@cs.ox.ac.uk
[2]University of Liverpool, UK
xiaowei.huang@liverpool.ac.uk

**Abstract**

Concolic testing alternates between CONCrete program execution and symbOLIC analysis to explore the execution paths of a software program and to increase code coverage. In this paper, we develop the first concolic testing approach for Deep Neural Networks (DNNs). More specifically, we utilise quantified linear arithmetic over rationals to express test requirements that have been studied in the literature, and then develop a coherent method to perform concolic testing with the aim of better coverage. Our experimental results show the effectiveness of the concolic testing approach in both achieving high coverage and finding adversarial examples.

## 1 Introduction

Deep neural networks (DNNs) have achieved great success in solving several long-standing tasks with near human-level intelligence, e.g., the ancient game of Go, image classification, and natural language processing. As a result, many potential applications are envisaged. However, major concerns have been raised about the readiness of applying this technique to safety- and security-critical systems, where faulty behaviour carries the risk of endangering human lives or potential damage to business. To address these concerns, similar to product development in avionics and automotive industries, a (safety or security) critical system implemented with DNNs, or comprising DNNs components, needs to be thoroughly tested and certified.

The software industry relies on testing as a primary means to provide stakeholders with information about the quality of the software product or service under test [1]. Existing efforts aimed at applying software testing techniques to DNNs are sparse, with a few recent attempts [2–6] either based on concrete

execution, e.g., Monte Carlo tree search [2] and gradient-based search [3, 4, 6], or symbolic execution, e.g., linear programming [5]. Together with these test-case generation algorithms, several test-coverage criteria have been presented, including neuron coverage [3], a criterion that is inspired by MC/DC [5], and criteria to capture particular neuron activation values to identify corner cases [6]. However, none of this work leverages *concolic testing* [7, 8], which combines concrete execution and symbolic analysis to explore the execution paths of a program that are hard to cover by other techniques such as random testing.

We hypothesize that concolic testing is particularly well-suited for DNNs. In a DNN, the input space is usually high-dimensional (e.g., a DNN for image classification takes as input tens of thousands of pixels), which makes random testing difficult. Moreover, due to the widespread use of the ReLU activation function for hidden neurons, the number of "execution paths" in a DNN is simply too large to be completely covered by symbolic execution. Concolic testing can mitigate the complexity by directing the symbolic analysis to particular execution paths, through concretely evaluating given properties of the DNN.

In this paper, we present the first concolic testing method for DNNs. Currently, test requirements for DNNs lack a unified format. To enable working with a broad spectrum of test requirements, we utilise Quantified Linear Arithmetic over Rationals (QLAR) to express them. For a given set $\mathfrak{R}$ of test requirements, we gradually generate test cases to improve coverage by alternating between concrete execution and symbolic analysis. Given an unsatisfied test requirement $r$, it is transformed into its corresponding form $\delta(r)$ by means of a heuristic function $\delta$. Then, for the current set $\mathcal{T}$ of test cases, we identify a pair $(t, \delta(r))$ of test case $t \in \mathcal{T}$ and requirement $\delta(r)$ such that $t$ is close to satisfying $r$ according to an evaluation based on *concrete execution*. After that, *symbolic analysis* is applied to $(t, \delta(r))$ to obtain a new concrete test case $t'$, which is then added to the existing test suite, i.e., $\mathcal{T} = \mathcal{T} \cup \{t'\}$. This process repeats until we reach a satisfactory level of coverage.

The generated test suite $\mathcal{T}$ is given to a *robustness oracle*, which detects whether $\mathcal{T}$ includes adversarial examples, i.e., test cases that have different classification labels when close to each other with respect to a distance metric. The lack of robustness has been viewed as a major weakness of DNNs, and the discovery of adversary examples [9] and the robustness problem are studied actively in several domains, including machine learning, automated verification, cyber security, and software testing.

Overall, the main contributions of this paper are threefold.

1. We develop the first concolic testing method for DNNs.

2. We utilise QLAR to express a set of safety-related properties, including Lipschitz continuity [2, 10–13] and several coverage metrics [3, 5, 6], and show that the new algorithm can work with these properties in a coherent way.

3. We implement the concolic testing method in the software tool DeepCon-

colic[1]. The experimental results show the effectiveness of DeepConcolic in not only achieving high coverage on the test requirements, but also discovering a significant proportion of adversarial examples.

# 2 Related Work

We briefly review existing efforts aimed at assessing the robustness of DNNs and the state of the art in concolic testing.

## 2.1 Robustness of DNNs

Current work on the robustness of DNNs can be categorised as offensive or defensive. Offensive approaches focus on heuristic search algorithms (mainly guided by the forward gradient or cost gradient of the DNN) to find adversarial examples that are as close as possible to a correctly classified input. On the other hand, the goal of defensive work is to increase the robustness of DNNs. There is an arms race between offensive and defensive techniques.

In this paper we focus on defensive methods. A promising approach is automated verification, which aims to provide robustness guarantees for DNNs. The main relevant techniques include performing a layer-by-layer exhaustive search [14], methods that use constraint solvers [15], and global optimisation approaches [13]. Exhaustive search suffers from the state-space explosion problem, which can be alleviated by Monte Carlo tree search [2]. Constraint-based approaches are limited to small DNNs with hundreds of neurons. Global optimisation improves over constraint-based approaches through its ability to work with large DNNs, but its capacity is sensitive to the number of input dimensions that need to be perturbed.

The application of traditional testing techniques to DNNs is difficult, and work that attempts to do so is more recent, e.g., [2–6]. Methods inspired by software testing methodologies typically employ coverage criteria to guide the generation of test cases; the resulting test suite is then searched for adversarial examples by querying an oracle. The coverage criteria considered include *neuron coverage* [3], which resembles traditional statement coverage. A set of criteria inspired by MD/DC coverage [16] is used in [5]; [6] presents criteria that are designed to capture particular values of neuron activations. [4] studies the utility of neuron coverage for detecting adversarial examples in DNNs for the Udacity driving challenge. In terms of the test case generation algorithms, [2] aims to cover the input space by exhaustive mutation testing and has theoretical guarantees, while in [3, 4, 6] gradient-based search algorithms are applied to solve optimisation problems, and [5] applies linear programming. None of these considers concolic testing and a general formalism for describing test requirements as we do in this paper.

[1]Available on GitHub: `https://github.com/TrustAI/DeepConcolic`

### 2.2 Concolic Testing

By concretely executing the program with particular inputs, e.g., random testing, a large number of inputs can often be easily tested. However, without deliberate design, the generated test cases may be restricted to a subset of the execution paths of a program and the chance of exploring the other execution paths that may contain bugs can be extremely low. In symbolic execution [17–19] an execution path is symbolically encoded. Modern-day constraint solvers ensure the feasibility of encoding and solving such symbolic representations, although the performance is still seriously affected by the size of the symbolic representations. Concolic testing [7, 8] is an effective approach to automated test case generation. It is a hybrid software technique that interleaves concrete execution, i.e., testing on particular inputs, with symbolic execution, a classical technique that treats program variables as symbolic ones [20].

Concolic testing has been applied routinely in software testing, with a wide range of tools available, e.g., [7, 8, 21]. It starts from executing the program with a concrete input. At the end of the concrete run, another execution path must be selected (by heuristics). This new execution path is then symbolically analysed, for example encoded and solved by a constraint solver, to yield a new concrete input. The concrete execution and symbolic analysis interleave until a certain level of coverage on program statements, branches, execution paths, etc., is reached.

The key factor affecting the performance of concolic testing is the heuristics used to select another execution path. While there are simple approaches such as random search and depth first search, more carefully designed heuristics can help achieve better coverage [21, 22]. Automated generation of search heuristics is an active thread of research in a few recent works [23, 24].

## 3 Deep Neural Networks

A (feedforward and deep) neural network, or DNN, is a tuple $\mathcal{N} = (L, T, \Phi)$ such that $L = \{L_k | k \in \{1, \ldots, K\}\}$ is a set of layers, $T \subseteq L \times L$ is a set of connections between layers, and $\Phi = \{\phi_k | k \in \{2, \ldots, K\}\}$ is a set of *activation functions*. Each layer $L_k$ consists of $s_k$ *neurons*, and the $l$th neuron of layer $k$ is denoted by $n_{k,l}$. We use $v_{k,l}$ to denote the value of $n_{k,l}$. Except for inputs, every neuron is connected to neurons in the preceding layer by pre-defined weights such that $\forall 1 < k \leq K, \forall 1 \leq l \leq s_k$,

$$v_{k,l} = \sum_{1 \leq h \leq s_{k-1}} \{w_{k-1,h,l} \cdot v_{k-1,h}\} + b_{k,l} \tag{1}$$

where $w_{k-1,h,l}$ is the pre-trained weight for the connection between $n_{k-1,h}$ (i.e., the $h$th neuron of layer $k-1$) and $n_{k,l}$ (i.e., the $l$th neuron of layer $k$), and $b_{k,l}$ is the *bias*.

Values of neurons in hidden layers (with $1 < k < K$) need to pass through a Rectified Linear Unit (ReLU) [25], such that *the final activation value* of each

neuron of hidden layers is

$$v_{k,l} = ReLU(v_{k,l}) = \begin{cases} v_{k,l} & \text{if } v_{k,l} > 0 \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

ReLU is by far the most popular and effective activation function for neural networks.

Finally, for any input, the neural network assigns a *label*, that is, the index of the neuron of the output layer having the largest value i.e., $label = \text{argmax}_{1 \leq l \leq s_K} \{v_{K,l}\}$.

Due to the existence of ReLU, the neural network is a highly non-linear function that approximates, e.g., the human perception ability. In this paper, we use variable $x$ to range over all possible inputs in the input fomain $D_{L_1}$ and use $t, t_1, t_2, ...$ to denote concrete inputs. Given a particular input $t$, we say that the DNN $\mathcal{N}$ is instantiated and we use $\mathcal{N}[t]$ to denote this instance of the network.

- Given a network instance $\mathcal{N}[t]$, the activation value of each neuron $n_{k,l}$ of the network and the final classification label are determined and denoted as $v[t]_{k,l}$ and $label[t]$ respectively. We write $v[t]_k$ for $1 \leq k \leq s_k$ to denote the vector of activations for neurons in layer $k$.
- When the input is given, the activation or deactivation of each ReLU operator in the DNN is determined.

We remark that, while for simplicity the definition focuses on DNNs with fully connected and convolutional layers, as shown in the experimental (Section 11), our method applies to other popular layers, e.g., maxpooling, used in the state-of-the-art DNNs.

# 4 Test Requirements

A software program has a set of concrete execution paths. Similarly, a DNN has a set of linear behaviours called *activation patterns* [5].

**Definition 1 (Activation Pattern)** *Given a network $\mathcal{N}$ and an input $t$, the activation pattern of $\mathcal{N}[t]$ is a function $ap[\mathcal{N}, t]$, mapping from the set of hidden neurons to $\{\mathit{true}, \mathit{false}\}$. We may write $ap[\mathcal{N}, t]$ as $ap[t]$ if $\mathcal{N}$ is clear from the context. For an activation pattern $ap[t]$, we use $ap[t]_{k,i}$ to denote whether the ReLU of the neuron $n_{k,i}$ is activated or not. Formally,*

$$\begin{array}{rcl} ap[t]_{k,l} = \mathit{false} & \equiv & u[t]_{k,l} < v[t]_{k,l} \\ ap[t]_{k,l} = \mathit{true} & \equiv & u[t]_{k,l} = v[t]_{k,l} \end{array} \tag{3}$$

The $u[t]_{k,l}$ and $v[t]_{k,l}$ denote the activation value of $n_{k,l}$ before and after the ReLU. Intuitively, $ap[t]_{k,l} = \mathsf{true}$, if the ReLU of the neuron $n_{k,l}$ is activated, and $ap[t]_{k,l} = \mathsf{false}$, otherwise.

Given a DNN instance $\mathcal{N}[t]$, each ReLU operator's behavior (i.e., each $ap[t]_{k,l}$) is fixed and this results in the particular activation pattern $ap[t]$, which can be encoded by using a Linear Programming (LP) model [5].

Finding a test suite to cover all activation patterns in a DNN is intractable. This is compounded by the fact that a real-world DNN can easily have millions of neurons. Therefore, we adopt the testing approach to first identify a subset of the activation patterns according to certain cover criteria or test requirements, and then generate test cases to cover these activation patterns.

## 4.1 Quantified Linear Arithmetic over Rationals (QLAR)

We adopt QLAR to express the *DNN Requirement*, abbreviated as DR. The benefit of a coherent logic framework is that it not only provides a general way of expressing requirements (Sections 5 and 7), but also enables the resolution of a class of problems with a small set of algorithms (Section 8).

**Definition 2** *We use variables $IV = \{x, x_1, x_2, ...\}$ to range over the inputs in $D_{L_1}$. Given a network $\mathcal{N}$, we let $V = \{u[x]_{k,l}, v[x]_{k,l} \mid 1 \leq k \leq K, 1 \leq l \leq s_k, x \in IV\}$ be a set of variables. DR uses the following syntax to write a requirement formula:*

$$\begin{array}{l} r ::= \ Qx.e \mid Qx_1, x_2.e \mid \arg opt_x a : e \mid \arg opt_{x_1,x_2} a : e \\ e ::= \ a \bowtie 0 \mid e \wedge e \mid \neg e \mid |\{e_1, ..., e_m\}| \bowtie q \\ a ::= w \mid c * w \mid p \mid a + a \mid a - a \end{array} \tag{4}$$

*where $Q \in \{\exists, \forall\}$, $w \in V$, $c, p \in \mathbb{R}$, $q \in \mathbb{N}$, $opt \in \{\max, \min\}$, $\bowtie \in \{\leq, <, =, >, \geq\}$, and $x, x_1, x_2 \in IV$. We may call $r$ a requirement formula, $e$ a Boolean formula, and $a$ an arithmetic formula. We call the logic $DR^{\exists}$ if the opt operators are not allowed, and $DR^{\exists,+}$ if both opt operators and the negation operator $\neg$ are not allowed.* We use $\mathfrak{R}$ to denote a set of requirement formulas.

Intuitively, formula $\exists x.r$ expresses that there exists an input $x$ such that $r$ is true, $\forall x.r$ expresses that $r$ is true for all inputs $x$, and $\arg\max_x a : e$ ($\arg\min_x a : e$, resp.) is to find the input $x$ among those satisfying Boolean formula $e$ to maximise (minimise) the value of the arithmetic formula $a$. The formulas $\exists x_1, x_2.r$, $\forall x_1, x_2.r$, $\arg\max_{x_1,x_2} e : r$, and $\arg\min_{x_1,x_2} e : r$ have similar meaning, except that they quantify over two input variables $x_1$ and $x_2$. Formula $|\{e_1, ..., e_m\}| \bowtie q$ expresses that the number of satisfiable Boolean expressions in the set $\{e_1, ..., e_m\}$ is in relation $\bowtie$ with $q$. Other operators in Boolean formulas and arithmetic formulas have standard meaning as in Boolean logic and linear programming.

Although $V$ does not include variables to express activation pattern $ap[x]$, we may write

$$ap[x_1]_{k,l} = ap[x_2]_{k,l} \text{ and } ap[x_1]_{k,l} \neq ap[x_2]_{k,l} \tag{5}$$

to express that $x_1$ and $x_2$ have, respectively, the same and different activation behaviour on neuron $n_{k,l}$. They can be expressed with Boolean formulas by

utilising variables in $V$ and the expressions in Equation (3). Moreover, some norm-based distances between two inputs can be expressed with a Boolean expression $e$. For example, we can use a set of requirements

$$\{x_1(i) - x_2(i) \leq q, x_2(i) - x_1(i) \leq q \mid i \in [1..s_1]\} \tag{6}$$

to express $||x_1 - x_2||_\infty \leq q$, the Chebyshev distance $L_\infty$ between two inputs $x_1$ and $x_2$, where $x(i)$ is the $i$th dimension of the input $x$.

## 4.2 Semantics

We define the satisfiability of a requirement $r$ over a test suite $\mathcal{T}$ containing a finite set of inputs. We assume that a variable will not be quantified more than once in a formula. For example, formula $\exists x.(e_1 \wedge \exists x.e_2)$ is not allowed, while $\exists x_1.(e_1 \wedge \exists x_2.e_2)$ is allowed. Those formulas in which variables are quantified more than once can be easily transformed into its legitimate form by variable substitutions.

**Definition 3** *Given a set $\mathcal{T}$ of test cases and a requirement $r$, the satisfiability relation $\mathcal{T} \models r$ is defined as follows.*

- $\mathcal{T} \models \exists x.e$ *if there exists some* $t \in \mathcal{T}$ *such that* $\mathcal{T} \models e[x \mapsto t]$, *where* $e[x \mapsto t]$ *is to substitute the occurences of* $x$ *with* $t$.
- $\mathcal{T} \models \exists x_1, x_2.e$ *if there exist two inputs* $t_1, t_2 \in \mathcal{T}$ *such that* $\mathcal{T} \models e[x_1 \mapsto t_1][x_2 \mapsto t_2]$
- $\mathcal{T} \models \arg\min_x a : e$ *returns an input* $t \in \mathcal{T}$ *such that,* $\mathcal{T} \models e[x \mapsto t]$ *and for all* $t' \in \mathcal{T}$ *such that* $\mathcal{T} \models e[x \mapsto t']$ *we have* $a[x \mapsto t] \leq a[x \mapsto t']$.
- $\mathcal{T} \models \arg\min_{x_1,x_2} a : e$ *returns two inputs* $t_1, t_1 \in \mathcal{T}$ *such that,* $\mathcal{T} \models e[x_1 \mapsto t_1][x_2 \mapsto t_2]$ *and for all* $t'_1, t'_2 \in \mathcal{T}$ *such that* $\mathcal{T} \models e[x_1 \mapsto t'_1][x_2 \mapsto t'_2]$ *we have* $a[x_1 \mapsto t_1][x_2 \mapsto t_2] \leq a[x_1 \mapsto t'_1][x_2 \mapsto t'_2]$.

*The cases for* $\arg\max$ *formulas are similar to those for* $\arg\min$, *by replacing* $\leq$ *with* $\geq$. *The cases for* $\forall$ *formulas are similar to those for* $\exists$ *in the standard way. Note that, when evaluating* $\arg opt$ *formulas (e.g.,* $\arg\min_x a : e$*), if an input* $t$ *is returned, we may need the value (*$\min_x a : e$*) as well. We use* $val(t, r)$ *to denote such a value for the returned input* $t$ *and the requirement formula* $r$. *For the evaluation of Boolean expression* $e$ *over an input* $t$, *we have*

- $\mathcal{T} \models a \bowtie 0$ *if* $a \bowtie 0$
- $\mathcal{T} \models e_1 \wedge e_2$ *if* $\mathcal{T} \models e_1$ *and* $\mathcal{T} \models e_2$
- $\mathcal{T} \models \neg e$ *if not* $\mathcal{T} \models e$
- $\mathcal{T} \models |\{e_1, ..., e_m\}| \bowtie q$ *if* $|\{e_i \mid \mathcal{T} \models e_i, i \in \{1, ..., m\}\}| \bowtie q$

*For the evaluation of arithmetic expression* $a$ *over an input* $t$,

- *$u[t]_{k,l}$ and $v[t]_{k,l}$ have their values from the activations of the DNN, $c{*}u[t]_{k,l}$ and $c * v[t]_{k,l}$ have the standard meaning for $c$ being the coefficient,*
- *$p$, $a_1 + a_2$, and $a_1 - a_2$ have the standard semantics.*

Similarly to Definition 3, we can define the semantics based on a satisfiability relation $X \models r$ where $X \subseteq D_{L_1}$ is a (maybe continuous) input subspace. Note that, while $\mathcal{T}$ is finite, $X$ may contain an infinite number of inputs. The relation $X \models r$ largely follows that of $\mathcal{T} \models r$ by replacing $\mathcal{T}$ with $X$. We have the following proposition.

**Proposition 1** *Given a $DR^{\exists,+}$ requirement $r$, a test suite $\mathcal{T}$ and a subspace $X \subseteq D_{L_1}$, if all test cases in $\mathcal{T}$ are also in $X$, we have that $X \models r$ implies $\mathcal{T} \models r$ but not vice versa.*

## 4.3 Test Criteria

Now we can define test criteria with respect to a set of test requirements and a set of test cases. When evaluating a criterion, we do not consider formulas with operators $\arg\mathit{opt}$, which are used primarily in Section 7 to express heuristics.

**Definition 4 (Test Criterion)** *Given a network $\mathcal{N}$, a set $\mathfrak{R}$ of test requirements expressed as DR formulas, and a test suite $\mathcal{T}$, the test criterion $M(\mathfrak{R}, \mathcal{T})$ is as follows:*

$$M(\mathfrak{R}, \mathcal{T}) = \frac{|\{r \in \mathfrak{R} \mid \mathcal{T} \models r\}|}{|\mathfrak{R}|} \tag{7}$$

Intuitively, it computes the percentage of the test requirements that are satisfied by test cases in $\mathcal{T}$. Similarly, we may define $M(\mathfrak{R}, X)$, called the true test criterion over $X$, for the consideration of the test requirement $\mathfrak{R}$ over all possible inputs in $X$. For $\mathcal{T} \subseteq X \subseteq D_{L_1}$, we have that

$$M(\mathfrak{R}, \mathcal{T}) \leq M(\mathfrak{R}, X) \leq M(\mathfrak{R}, D_{L_1}) = 1.0 \tag{8}$$

when all requirements in $\mathfrak{R}$ are $\mathrm{DR}^{\exists,+}$ formulas.

## 4.4 Computational Complexity

We study the computational complexity of the test requirements satisfaction problem. For the testing, it is in polynomial time with respect to the number of test cases in the test suite.

**Theorem 1** *Given a network $\mathcal{N}$, a DR requirement formula $r$ with a constant number of $\exists$, $\forall$, $\arg\max$, $\arg\min$ operators, and a test suite $\mathcal{T}$, the checking of $\mathcal{T} \models r$ can be done in polynomial time with respect to the size of $\mathcal{T}$.*

However, the general verification problem is NP-complete with respect to the number of hidden neurons.

**Theorem 2** *Given a network $\mathcal{N}$, a DR requirement formula $r$ with a constant number of* $\exists$, $\arg\max$, $\arg\min$ *operators, and a subspace $X \subseteq D_{L_1}$, the checking of $X \models r$ is NP-complete. This conclusion also holds for $DR^{\exists}$ and $DR^{\exists,+}$ requirements.*

# 5 Concrete Requirements

In this section, we use $\mathrm{DR}^{\exists,+}$ formulas to express several important requirements for DNNs, including Lipschitz continuity [2, 10–13] and test criteria [3, 5, 6]. Test criteria to be considered here bear syntactical similarity with the test criteria in software testing. Lipschitz continuity is semantic, specific to DNNs, and shown to be closely related to the theoretical understanding of convolutional DNNs [10] and the robustness of both DNNs [2, 13] and Generative Adversarial Networks [11]. These requirements have been studied in the literature using different formalisms and approaches.

Each test criterion specifies a set of test requirements, the cover of which by test cases shall enforce a certain level of confidence for the safety of the DNN under testing. In the following, we study three example test criteria from [3, 5, 6], respectively. We use $||t_1 - t_2||_q$ to denote the distance between two inputs $t_1$ and $t_2$ with respect to a distance metric $||\cdot||_q$. The metric $||\cdot||_q$ can be, e.g., a norm-based metric such as the $L_0$-norm (Hamming distance), $L_2$-norm (Euclidean distance), and $L_\infty$-norm (Chebyshev distance), or a structural similarity distance, such as SSIM [26]. In the following, we fix a distance metric and simply write $||t_1 - t_2||$. Section 11 will give the metrics we use for the experiments.

We may consider requirements for a set of input subspaces. Given a real number $b$, we can generate a finite set $\mathcal{S}(D_{L_1}, b)$ of subspaces of $D_{L_1}$ such that for all inputs $x_1, x_2 \in D_{L_1}$, if $||x_1 - x_2|| \leq b$ then there exists a subspace $X \in \mathcal{S}(D_{L_1}, b)$ such that $x_1, x_2 \in X$. The subspaces can be overlapping. Usually, every subspace $X \in \mathcal{S}(D_{L_1}, b)$ can be represented with a box constraint, e.g., $X = [l, u]^{s_1}$, and therefore $t \in X$ can be expressed with a Boolean expression as follows.

$$\bigwedge_{i=1}^{s_1} x(i) - u \leq 0 \wedge x(i) - l \geq 0 \tag{9}$$

## 5.1 Lipschitz Continuity

Lipschitz continuity has been shown in [9, 13] to hold for a large class of DNNs, including, e.g., image classification DNNs.

**Definition 5 (Lipschitz Continuity)** *A network $\mathcal{N}$ is called* Lipschitz continuous *if there exists a real constant $c \geq 0$ such that, for all $x_1, x_2 \in D_{L_1}$:*

$$||v[x_1]_1 - v[x_2]_1|| \leq c * ||x_1 - x_2|| \tag{10}$$

*The value $c$ is called the* Lipschitz constant*, and the smallest $c$ is called the* best Lipschitz constant*, denoted as $c_{best}$. Recall that $v[x]_1$ denotes the vector of activations for neurons at the input layer.*

Since the computation of $c_{best}$ is an NP-hard problem and a smaller $c$ can significantly improve the performance of verification algorithms [2, 13], it is interesting to know whether a given number $c$ is a legitimate Lipschitz constant, either for the entire input space $D_{L_1}$ or for some subspace $X \in \mathcal{S}(D_{L_1}, b)$. The testing of Lipschitz continuity can be guided by having the following requirements.

**Definition 6 (Lipschitz Requirements)** *Given a real number $c > 0$ and an integer $b > 0$, a set $\mathfrak{R}_{Lip}(b, c)$ of Lipschitz requirements is*

$$\begin{array}{c}\{\exists x_1, x_2.(||v[x_1]_1 - v[x_2]_1|| - c * ||x_1 - x_2|| > 0) \\ \wedge x_1, x_2 \in X \mid X \in \mathcal{S}(D_{L_1}, b)\}\end{array} \tag{11}$$

Intuitively, for each $X \in \mathcal{S}(D_{L_1}, b)$, this requirement expresses the existence of two inputs $x_1$ and $x_2$ such that $\mathcal{N}$ breaks the Lipschitz constant $c$. From the perspective of cover, given a number $c$, the true test criteria $M(\mathfrak{R}_{Lip}(b, c), D_{L_1})$ may be impossible to reach 100%, because there may exist $r \in \mathfrak{R}_{Lip}(b, c)$ such that $D_{L_1} \not\models r$. However, for a test case generation algorithm, the goal is to produce $\mathcal{T}$ so that, although $M(\mathfrak{R}_{Lip}(b, c), \mathcal{T}) \leq M(\mathfrak{R}_{Lip}(b, c), D_{L_1})$, their gap is minor.

## 5.2 Neuron Cover

The Neuron Cover (NC) [3] is an adaptation of the statement cover in software testing. Its definition is as follows.

**Definition 7** *The neuron cover for a DNN $\mathcal{N}$ is to find a testsuite $\mathcal{T}$ of inputs such that, for any hidden neuron $n_{k,i}$, there exists test case $t \in \mathcal{T}$ such that $ap[t]_{k,i} =$ true.*

This can be expressed with the following requirements in $\mathfrak{R}_{NC}$, each of which requests that for the neuron $n_{k,i}$, there is an input $x$ that activates the neuron, i.e., $ap[x]_{k,i} =$ true.

**Definition 8 (NC Requirements)** *The set $\mathfrak{R}_{NC}$ of requirements is*

$$\{\exists x.ap[x]_{k,i} = \textit{true} \mid 2 \leq k \leq K-1, 1 \leq i \leq s_k\} \tag{12}$$

## 5.3 Modified Condition/Decision Cover (MC/DC)

In [5], a family of four test criteria are proposed, inspired by the MC/DC in software testing, and here we work with the Sign-Sign Cover (SSC) of it. According to [5], each neuron $n_{k+1,j}$ can be regarded as a decision such that these neurons at the precedent layer (i.e., the $k$-th layer) are conditions that define its activation value, as in Equation (1). By adapting MC/DC into DNNs,

it means that each condition neuron must be shown to independently affect the outcome of the decision neuron. In particular, the SSC observes the change of a decision or condition neuron, if the sign of its activation, which is either positive or negative, changes.

Consequently, the test requirements for SSC are defined by the following set.

**Definition 9 (SSC Requirements)** *Given a pair $\alpha = (n_{k,i}, n_{k+1,j})$ of neurons, the singleton set $\mathfrak{R}_{SSC}(\alpha)$ of requirements is as follows:*

$$\begin{gathered}\{\exists x_1, x_2.\ ap[x_1]_{k,i} \neq ap[x_2]_{k,i} \wedge ap[x_1]_{k+1,j} \neq ap[x_2]_{k+1,j} \wedge \\ \textstyle\bigwedge_{1 \leq l \leq s_k, l \neq i} ap[x_1]_{k,l} - ap[x_2]_{k,l} = 0\}\end{gathered} \tag{13}$$

*and we have*

$$\mathfrak{R}_{SSC} = \bigcup_{2 \leq k \leq K-2, 1 \leq i \leq s_k, 1 \leq j \leq s_{k+1}} \mathfrak{R}_{SSC}((n_{k,i}, n_{k+1,j})) \tag{14}$$

That is, for each pair $(n_{k,i}, n_{k+1,j})$ of neurons at two adjacent layers $k$ and $k+1$ respectively, we need two inputs $x_1$ and $x_2$ such that the sign change of $n_{k,i}$ independently affects the sign change of $n_{k+1,j}$. Other neurons at layer $k$ are required to maintain their signs between $x_1$ and $x_2$ to ensure the independent affection. The idea of SS Cover (and all other test criteria in [5]) is to ensure that not only the presence of a feature needs to be tested but also the effects of less complex features on a more complex feature must be tested.

### 5.4 Neuron Boundary Cover

The Neuron Boundary Cover (NBC) [6] aims to cover neuron activation values that exceed pre-specified bounds. It can be formulated as follows.

**Definition 10 (Neuron Boundary Cover Requirements)** *Given two sets of bounds $h = \{h_{k,i}\}_{2 \leq k \leq K-1, 1 \leq i \leq s_k}$ and $l = \{l_{k,i}\}_{2 \leq k \leq K-1, 1 \leq i \leq s_k}$, the set $\mathfrak{R}_{NBC}(h, l)$ of requirements is*

$$\begin{gathered}\{\exists x.\ u[x]_{k,i} - h_{k,i} > 0,\ \exists x.\ u[x]_{k,i} - l_{k,i} < 0 \mid \\ 2 \leq k \leq K-1, 1 \leq i \leq s_k\}\end{gathered} \tag{15}$$

*where $h_{k,i}$ and $l_{k,i}$ are the upper and lower bounds on the activation value of a neuron $n_{k,i}$.*

## 6 Overall Design

This section describes the overall design of the concolic testing approach for requirements expressed using our formalism. The method alternates between concretely evaluating a DNN's activation patterns and symbolically generating new inputs. The concolic testing pseudocode is in Algorithm 1 and the corresponding workflow is depicted in Figure 1.

Algorithm 1 takes as inputs a DNN $\mathcal{N}$, an input $t_0$, a heuristic $\delta$, and a set $\mathfrak{R}$ of requirements, and produces a test suite $\mathcal{T}$. In the algorithm, $t$ is the latest test case generated, and is initialised as the input $t_0$. For every test requirement $r \in \mathfrak{R}$, it is removed from $\mathfrak{R}$ whenever satisfied by $\mathcal{T}$, i.e., $\mathcal{T} \models r$.

The function *requirement_evaluation* (Line 7), whose details are given in Section 7, aims to find a pair $(t, \delta(r))$[2] of input and requirement which, according to our concrete evaluation, are the most promising in finding a new test case $t'$ to satisfy the requirement $r$. The heuristic $\delta$ is a transformation function mapping a quantified formula $r$ with operator $\exists$ into an optimisation formula $\delta(r)$ with operator $\arg opt$. In the evaluation, concrete executions are applied on the current test suite $\mathcal{T}$ and the transformed requirements $\delta(\mathfrak{R})$, according to the semantics given in Definition 3.

After obtaining $(t, \delta(r))$, a symbolic analysis approach *symbolic_analysis* (Line 8), whose details are in Section 8, is applied to have a new concrete input $t'$. Then a function *validity_check* (Line 9), whose details are given in Section 9, is applied to check if the new input is valid or not. The set $S$ maintains a set of $(t, r)$ pairs on which our symbolic analysis cannot find a valid new input.

The algorithm has two termination conditions. When all test requirements in $\mathfrak{R}$ have been satisfied, i.e., $\mathfrak{R} = \emptyset$, or no further requirement in $\mathfrak{R}$ can be satisfied, i.e., $S \cup \mathcal{T} = \mathcal{T} \times \mathfrak{R}$, the algorithm terminates and returns the current test suite $\mathcal{T}$.

**Algorithm 1** Concolic Testing Algorithm for DNNs

**INPUT:** $\mathcal{N}, \mathfrak{R}, \delta, t_0$
**OUTPUT:** $\mathcal{T}$

```
1:  T ← {t_0} and S = {}
2:  t ← t_0
3:  while R ≠ ∅ do
4:    for each r ∈ R do
5:      if T ⊨ r then R ← R \ {r}
6:    while true do
7:      t, δ(r) ← requirement_evaluation(T, δ(R))
8:      t' ← symbolic_analysis(t, δ(r))
9:      if validity_check(t') = true then
10:       T ← T ∪ {t'}
11:       break
12:     else
13:       S ← S ∪ {(t, r)}
14:       if S = T × R then return T
15: return T
```

As shown in Figure 1, after the generation of test suite $\mathcal{T}$ by Algorithm 1,

[2]For some requirements, we might return two inputs $t_1$ and $t_2$. Here, for simplicity, we describe the case for a single input. The generalisation to two inputs is straightforward.

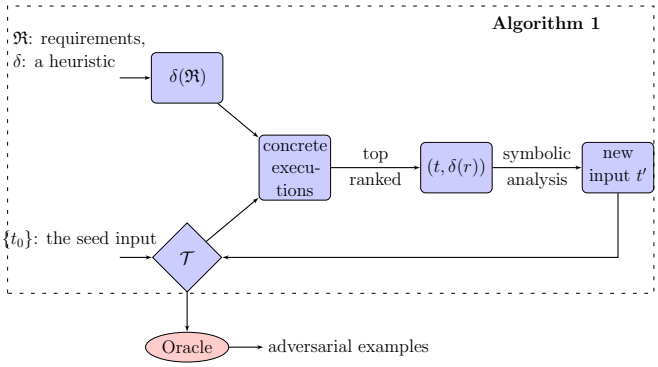



Figure 1: The work flow for our concolic testing.

$\mathcal{T}$ will run through an oracle, i.e., *robustness_oracle* in Section 9, in order to evaluate the robustness of the DNN.

# 7 Requirement Evaluation

This section presents our approach for Line 7 of Algorithm 1. Given a set of requirements $\mathfrak{R}$ that have not been satisfied, a heuristic $\delta$, and the current set $\mathcal{T}$ of test cases, the goal is to select a concrete input $t \in \mathcal{T}$ together with a requirement $r' = \delta(r)$ for some $r \in \mathfrak{R}$, both of which will be used later in a symbolic approach to find the next concrete input $t'$ (to be given in Section 8).

The general idea of obtaining $(t, \delta(r))$ is as follows. For all requirements $r \in \mathfrak{R}$, we transform $r$ into $\delta(r)$ by utilising operators $\arg opt$ for $opt \in \{\max, \min\}$. Then for all pairs $(t, \delta(r)) \in \mathcal{T} \times \delta(\mathfrak{R})$, we apply the semantics in Definition 3 to obtain an evaluation $val(t, \delta(r))$. As $\mathfrak{R}$ may contain more than one requirement, we return the pair $(t, \delta(r))$ such that

$$r = \arg\max_{r} \{val(t, \delta(r)) \mid r \in \mathfrak{R}\}. \tag{16}$$

## 7.1 Heuristics

For the several formulas $r$ discussed in Section 5, we present the requirement $\delta(r)$ used in this paper. We remark that, since $\delta$ is a heuristic, there exist other definitions. The following definitions work well in our experiments.

### 7.1.1 Lipschitz Continuity

When a Lipschitz requirement $r$ as in Equation (11) is unsatisfiable on $\mathcal{T}$, we transform it into $\delta(r)$ as follows.

$$\arg\max_{x_1, x_2} . ||v[x_1]_1 - v[x_2]_1|| - c * ||x_1 - x_2|| : x_1, x_2 \in X \tag{17}$$

According to the semantics in Definition 3, the aim is to find the best $t_1$ and $t_2$ from $\mathcal{T}$ to make the evaluation of $||v[t_1]_1 - v[t_2]_1|| - c * ||t_1 - t_2||$ as large

as possible. As described, we also need to compute $val(t_1, t_2, r) = ||v[t_1]_1 - v[t_2]_1|| - c * ||t_1 - t_2||$.

#### 7.1.2 Neuron Cover

When a requirement $r$ in Equation (12) is unsatisfiable on $\mathcal{T}$, we transform it into the following requirement $\delta(r)$:

$$\arg\max_x c_k \cdot u_{k,i}[x] : \mathsf{true} \tag{18}$$

According to the semantics, the requirement will return the input $t \in \mathcal{T}$ that has the maximal value for $c_k \cdot u_{k,i}[x]$.

The coefficient $c_k$ is a layer-wise constant. It is based on the following observation. With the propagation of signals in the DNN, activation values at each layer can be of different magnitudes. For example, if the minimum activation value of neurons at layer $k$ and $k+1$ are -10 and -100 respectively, then even when a neuron $u[x]_{k,i} = -1 > -2 = u[x]_{k+1,j}$, we may still regard $n_{k+1,j}$ as being closer to be activated than $u_{k,i}$ is. Consequently, we define a layer factor $c_k$ for each layer which normalises the average activation valuations of neurons at different layers into the same magnitude level.

#### 7.1.3 SS Cover

In the SS Cover, given a decision neuron $n_{k+1,j}$, the concrete evaluation aims to select one of its condition neurons $n_{k,i}$ at layer $k$ such that, for the test case to be generated, the signs of $n_{k,i}$ and $n_{k+1,j}$ can be negated and the rest of $n_{k+1,j}$'s condition neurons reserve their respective signs. This is achieved by having the following $\delta(r)$.

$$\arg\max_x -|u[x]_{k,i}| : \mathsf{true} \tag{19}$$

Intuitively, given the decision neuron $n_{k+1,j}$, Equation (19) selects the condition that is closest to the change of activation sign (i.e., smallest $|u[x]_{k,i}|$).

#### 7.1.4 Neuron Boundary Cover

We transform the requirement $r$ in Equation (20) into the following $\delta(r)$ when it is not satisfiable on $\mathcal{T}$; it selects the neuron that is closest to either the higher or lower boundary.

$$\begin{array}{l} \arg\max_x u[x]_{k,i} - h_{k,i} : \mathsf{true} \\ \arg\max_x l_{k,i} - u[x]_{k,i} : \mathsf{true} \end{array} \tag{20}$$

## 8 Symbolic Generation of New Concrete Inputs

This section presents our approach for Line 8 of Algorithm 1. That is, given a concrete input $t$ and a transformed requirement $r' = \delta(r)$, we need to find the next concrete input $t'$ by symbolic analysis. This new $t'$ will be added into the test suite, i.e., Line 10 of Algorithm 1. The symbolic analysis techniques to be considered

include the linear programming method in [5], a global optimisation method for the $L_0$ norm in [27], and a new optimisation algorithm to be introduced below. We regard optimisation algorithms as symbolic analysis methods because, similarly to constraint solving methods, they work with a set of test cases in a single run.

Thanks to the use of $DR$, for each symbolic analysis method, its application to different test criteria can be formulated under a unified logic framework. To ease the presentation, the following description may, for each algorithm, focus on a few requirements, but we remark that all algorithms can work with all the requirements given in Section 5.

## 8.1 Symbolic Analysis using Linear Programming (LP)

As explained in Section 4, given an input $x$, the DNN instance $\mathcal{N}[x]$ maps to an activation pattern $ap[x]$ that can be modeled using Linear Prorgamming (LP). Particularly, the following linear constraints synthesize a set of inputs that exhibit the same ReLU behaviour as $x$.

$$\begin{array}{l}\{\mathbf{u_{k,i}} \geq 0 \wedge \mathbf{v_{k,i}} = \mathbf{u_{k,i}} \mid ap[x]_{k,i} \geq 0, k \in [2,K), i \in [1..s_k]\} \\ \cup\{\mathbf{u_{k,i}} < 0 \wedge \mathbf{v_{k,i}} = 0 \mid ap[x]_{k,i} < 0, k \in [2,K), i \in [1..s_k]\}\end{array}$$

$$\{\mathbf{u}_{k,i} = \sum_{1 \leq j \leq s_{k-1}} \{w_{k-1,j,i} \cdot \mathbf{v}_{k-1,j}\} + \delta_{k,i} \mid k \in [2,K], i \in [1..s_k]\}$$

**Real valued variables** in the LP model are emphasized in **bold**. More encoding details can be found in [5].

Subsequently, the symbolic analysis for finding a new input $t'$ from a pair $(t, r')$ of input and requirement is equivalent to finding a new activation pattern. Note that, to make sure that the obtained test case is meaningful, in the LP model an objective is added to minimize the distance between $t$ and $t'$. Thus, the use of LP requires that a distance metric be linear, e.g., $L_\infty$-norm.

### 8.1.1 Neuron Cover

The symbolic analysis of neuron cover takes the input test case $t$ and requirement $r'$, let us say, on the activation of neuron $n_{k,i}$, and it shall return a new test $t'$ such that the test requirement is satisfied by the network instance $\mathcal{N}[t']$. As a result, given $\mathcal{N}[t]$'s activation pattern $ap[t]$, we can build up a new activation pattern $ap'$ such that

$$\{ap'_{k,i} = \neg ap[t]_{k,i} \wedge \forall k_1 < k : \bigwedge_{0 \leq i_1 \leq s_{k_1}} ap'_{k_1,i_1} = ap[t]_{k_1,i_1}\}$$

This activation pattern specifies the following conditions.

- $n_{k,i}$'s activation sign is negated: this ensures the aim of the symbolic analysis to activate $n_{k,i}$.

- In the new activation pattern $ap'$, the neurons before layer $k$ preserve their activation signs as in $ap[t]$. Though there may exist various activation patterns that make $n_{k,i}$ activated, for the use of LP modeling, one particular combination of activation signs must be pre-determined.
- Other neurons are irrelevant, as the sign of $n_{k,i}$ is only affected by the activation values of those neurons in previous layers.

Finally, by applying the LP modeling in [5] to the activation pattern defined in (8.1.1), and if there exists a feasible solution, then it will become the new test case $t'$, which makes the DNN satisfy the requirement $r'$.

### 8.1.2 SS Cover

When it comes to the SS Cover, to satisfy the requirement $r'$ we need to find a new test case such that, with respect to the input $t$, activation signs of $n_{k+1,j}$ and $n_{k,i}$ are negated, while other signs of other neurons at layer $k$ are kept the same as in the case of input $t$. To achieve this, the following activation pattern $ap'$ is built up for the LP modeling.

$$\begin{array}{c}\{ap'_{k,i} = \neg ap[t]_{k,i} \wedge ap'_{k+1,j} = \neg ap[t]_{k+1,j} \\ \wedge \forall k_1 < k : \bigwedge_{1 \le i_1 \le s_{k_1}} ap'_{k_1,i_1} = ap[t]_{k_1,i_1}\}\end{array}$$

### 8.1.3 Neuron Boundary Cover

In case of the neuron boundary cover, the symbolic analysis aims to find an input $t'$ such that the neuron $n_{k,i}$'s activation value exceeds either its higher bound $h_{k,i}$ or its lower bound $l_{k,i}$. To achieve this, while preserving the DNN activation pattern as $ap[t]$, we add one of the following constraints into the LP program.

- If $u[x]_{k,i} - h_{k,i} > l_{k,i} - u[x]_{k,i}$: $\mathbf{u_{k,i}} > h_{k,i}$;
- otherwise: $\mathbf{u_{k,i}} < l_{k,i}$.

## 8.2 Symbolic Analysis using Global Optimization

The symbolic analysis for finding a new input can also be implemented by solving the global optimization problem in [27]. That is, by specifying the test requirement as an optimization objective, we apply global optimization to find a test case that makes the test requirement satisfied. Readers are referred to [27] for the details of the algorithm.

- For the Neuron Cover, the objective is thus to find a $t'$ such that the specified neuron $n_{k,i}$ has $ap[t']_{k,i} =$true.

- In case of the SS Cover, given the neuron pair $(n_{k,i}, n_{k+1,j})$ and the original input $t$, the optimization objective becomes

$$ap[t']_{k,i} \neq ap[t]_{k,i} \wedge ap[t']_{k+1,j} \neq ap[t]_{k+1,j} \wedge \bigwedge_{i' \neq i} ap[t']_{k,i'} = ap[t]_{k,i}$$

- When it comes to the Neuron Boundary Cover, depending on whether the higher bound or lower bound for the activation of $n_{k,i}$ is considered, the objective of finding a new input $t'$ can be one of the two forms: 1) $u[t']_{k,i} > h_{k,i}$ or 2) $u[t']_{k,i} < l_{k,i}$.

## 8.3 Algorithms for Lipschitz Test Case Generation

Given a requirement in Equation (11) for a subspace $X$, we let $t_0 \in \mathbb{R}^n$ be the representative point of the subspace $X$ to which $t_1$ and $t_2$ belong. The optimisation problem is to generate two inputs $t_1$ and $t_2$ such that

$$\begin{aligned} &||v[t_1]_1 - v[t_2]_1||_{D_1} - c * ||t_1 - t_2||_{D_1} > 0 \\ &s.t. \;\; ||t_1 - t_0||_{D_2} \leq \Delta, \;\; ||t_2 - t_0||_{D_2} \leq \Delta \end{aligned} \tag{21}$$

where $||*||_{D_1}$ and $||*||_{D_2}$ denote certain norm metrics such as the $L_0$-norm, $L_2$-norm or $L_\infty$-norm, and $\Delta$ intuitively represents the radius of a norm ball (for $L_1, L_2$-norm) or the size of a hypercube (for $L_\infty$-norm) centered on $t_0$. $\Delta$ is a hyper-parameter of the algorithm.

The above problem can be efficiently solved by a novel *alternating compass search* scheme. Specifically, we alternately optimize the following two optimization problems through relaxation [28], i.e., maximizing the lower bound of the original Lipschitz constant instead of directly maximizing the Lipschitz constant itself. To do so we formulate the original non-linear proportional optimization as a linear problem when both norm metrics $||*||_{D_1}$ and $||*||_{D_2}$ are $L_\infty$-norm.

### 8.3.1 Stage One

we solve

$$\begin{aligned} \min_{t_1} F(t_1, t_0) = -||v[t_1]_1 - v[t_0]_1||_{D_1} \\ \text{s.t.} \;\; ||t_1 - t_0||_{D_2} \leq \Delta \end{aligned} \tag{22}$$

The above objective enables the algorithm to search for an optimal $t_1$ in the space of a norm ball or hypercube centered on $t_0$ with radius $\Delta$, such that the norm distance of $v[t_1]_1$ and $v[t_0]_1$ is as large as possible. From the constraint, we know that $\sup_{||t_1 - t_0||_{D_2} \leq \Delta} ||t_1 - t_0||_{D_2} = \Delta$. Thus a smaller $F(t_1, t_0)$ essentially leads to a larger Lipschitz constant, considering that $\mathbf{Lip}(t_1, t_0) = -F(t_1, t_0)/||t_1 - t_0||_{D_2} \geq -F(t_1, t_0)/\Delta$, i.e., $-F(t_1, t_0)/\Delta$ is the lower bound of $\mathbf{Lip}(t_1, t_0)$. Therefore, the searching trace of minimizing $F(t_1, t_0)$ will generally lead to an increase of the Lipschitz constant.

To solve the above the problem, we use the compass search method [29], which is efficient, derivative-free, and guaranteed to have the first-order global convergence. Because we aim to find an input pair to break the predefined Lipschitz constant $c$ instead of finding the largest Lipschitz constant, along each iteration, when we get $\bar{t}_1$, we check whether $\mathbf{Lip}(\bar{t}_1, t_0) > c$. If it holds, we find an input pair $\bar{t}_1$ and $t_0$ that satisfies the test requirement; otherwise, we continue the compass search until convergence or a satisfiable input pair is generated. If Equation (22) is convergent and we can find an optimal $t_1$ as

$$t_1^* = \arg\min_{t_1} F(t_1, t_0) \quad \text{s.t. } ||t_1 - t_0||_{D_2} \leq \Delta$$

but we still cannot find a satisfiable input pair, we perform Stage Two optimization.

#### 8.3.2 Stage Two

we solve

$$\begin{aligned} \min_{t_2} F(t_1^*, t_2) = -||v[t_2]_1 - v[t_1^*]_1||_{D_1} \\ \text{s.t. } \ ||t_2 - t_0||_{D_2} \leq \Delta \end{aligned} \tag{23}$$

Similarly, we use derivative-free compass search to solve the above problem and check whether $\mathbf{Lip}(t_1^*, t_2) > c$ holds at each iterative optimization trace $\bar{t}_2$. If it holds, we return the image pair $t_1^*$ and $\bar{t}_2$ that satisfies the test requirement; otherwise, we continue the optimization until convergence or a satisfiable input pair is generated. If Equation (23) is convergent at $t_2^*$, and we still cannot find such a input pair, we modify the objective function again by letting $t_1^* = t_2^*$ in Equation (23) and continue the search and satisfiability checking procedure.

#### 8.3.3 Stage Three

If the function $\mathbf{Lip}(t_1^*, t_2^*)$ stops increasing in Stage Two, we treat the whole search procedure as convergent and fail to find an input pair that can break the predefined Lipschitz constant $c$. In this case, we return the best input pair we can find, i.e., $t_1^*$ and $t_2^*$, and the largest Lipschitz constant $\mathbf{Lip}(t_1^*, t_2)$. Note that the returned constant is smaller than $c$.

In summary, the proposed method is an alternating optimization scheme based on the compass search. Basically, we start from the given $t_0$ to search for an image $t_1$ in a norm ball or hypercube (the optimization trajectory on the norm ball space is denoted as $S(t_0, \Delta(t_0))$) such that $Lip(t_0, t_1) > c$ (this step is symbolic execution); if we cannot find it, we modify the optimization objective function by replacing $t_0$ with $t_1^*$ (the best concrete input found in this optimization trace) to initiate another optimization trajectory on the space, i.e., $S(t_1^*, \Delta(t_0))$. This process is repeated until the optimization trace gradually covers the whole norm ball space $S(\Delta(t_0))$.

Table 1: Comparison between different coverage-based DNN testing methods

| | **DeepConcolic** | DeepXplore [3] | DeepTest [4] | DeepCover [5] | DeepGauge [6] |
|---|---|---|---|---|---|
| Coverage criteria | NC, SSC, NBC etc. | NC | NC | MC/DC | NBC etc. |
| Test generation | concolic | dual-optimisation | greedy search | symbolic execution | gradient descent methods |
| DNN inputs | single | multiple | single | single | single |
| Image inputs | single/multiple | multiple | multiple | multiple | multiple |
| Distance metric | $L_\infty, L_0$-norm | $L_1$-norm | Jaccard distance | $L_\infty$-norm | $L_\infty$-norm |

# 9 Oracle

First of all, we provide details to Line 10 of Algorithm 1 about the validity checking.

**Definition 11 (Validity Checking)** *Given a set $O$ of correctly classified inputs (e.g., the training dataset) and a real number $b$, a test case $t' \in \mathcal{T}$ passes the validity checking if*

$$\exists t \in O : \quad ||t - t'|| \leq b \tag{24}$$

Intuitively, it says that the test case $t$ is valid if it is close to some of the correctly classified inputs in $O$. Given a test case $t' \in \mathcal{T}$, we can write $O(t')$ for the input $t \in O$ which has the smallest distance to $t'$ among all inputs in $O$.

When checking the quality of the generated test suite $\mathcal{T}$, we use the following oracle.

**Definition 12 (Robustness Oracle)** *Given a set $O$ of correctly classified inputs, a test case $t'$ passes the robustness oracle if*

$$\arg\max_j v[t']_{K,j} = \arg\max_j v[O(t')]_{K,j} \tag{25}$$

Intuitively, the role of this oracle is to check the robustness of the DNN on input $O(t')$: if $t'$ cannot pass the oracle then it serves as evidence of the DNN lacking in robustness.

# 10 A Summary of Coverage-based DNN Testing

We briefly summarise the similarities and differences between our concolic testing method, named *DeepConcolic*, and other existing coverage-driven DNN testing methods: DeepXplore [3], DeepTest [4], DeepCover [5], and DeepGauge [6]. The details are presented in Table 1, where NC, SSC, and NBC are short for Neuron Cover, SS Cover, and Neuron Boundary Cover, respectively. In addition to the concolic nature of DeepConcolic, we observe the following differences.

- DeepConcolic is generic, using a unified language DR to express test requirements and a small set of algorithms to compute a class of requirements; the other methods are *ad hoc* tests for specific requirements.
- Comparing with DeepXplore, which needs a set of DNNs to explore multiple gradient directions, the other methods, including DeepConcolic, need a single DNN only.

Table 2: Neuron coverage of DeepConcolic and DeepXplore

| | **DeepConcolic** | | DeepXplore | | |
|---|---|---|---|---|---|
| | $L_\infty$-norm | $L_0$-norm | light | occlusion | blackout |
| MNIST | 97.89% | 97.24% | 80.5% | 82.5% | 81.6% |
| CIFAR-10 | 89.59% | 99.69% | 77.9% | 86.8% | 89.5% |

- In contrast to the other methods, DeepConcolic can achieve good coverage by starting from a single input; the other methods need a non-trivial set of inputs.
- DeepConcolic is can be parameterized with a desired norm distance metric $||\cdot||$.

Moreover, from the workflow given in Figure 1, we can see that DeepConcolic features a clean separation between the generation of test cases and the oracle. This is well aligned with the traditional approach to test case generation. The other methods essentially use the oracles of Section 9 as part of their objectives to guide the generation of test cases.

# 11 Experimental Results

The concolic testing approach in this paper has been implemented into a software tool DeepConcolic[3]. In this section, we compare it with the state-of-the-art DNN testing tool and evaluate its performance for different test requirements.

## 11.1 Comparison with DeepXplore

This section compares the use of DeepConcolic and DeepXplore[4] for two DNNs on the MNIST and CIFAR-10 datasets, respectively. The DNN model architecture is given in Table 4 (Appendix A.2), and the configurations for DeepXplore are in Appendix A.1.

### 11.1.1 Neuron Cover Results

For each tool, we start the neuron cover testing from a randomly sampled image input. Table 2 gives the neuron cover reports from the two tools. It can be seen that DeepConcolic returns the higher neuron coverage, *i.e.*, 15.39% higher on MNIST and 10.19% higher on CIFAR-10, for all the three image constraints ('light', 'occlusion', and 'blackout') that DeepXplore imposes on the image.

[3]The implementation and all data in this section are available on GitHub: `https://github.com/TrustAI/DeepConcolic`

[4]All the data for DeepXplore are generated by using the software package available from `https://github.com/peikexin9/deepxplore`.

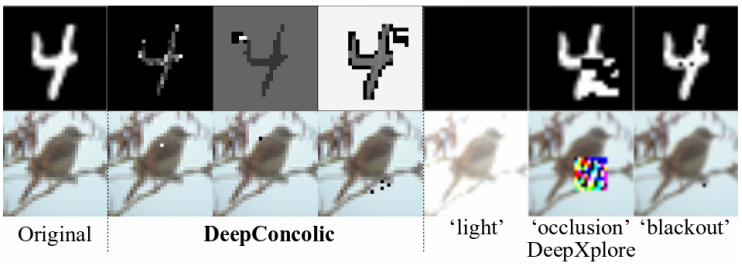



Figure 2: Adversarial images of DNNs, with $L_\infty$-norm for MNIST (top row) and $L_0$-norm for CIFAR10 (bottom row), generated by DeepConcolic and DeepXplore, the latter with image constraints 'light', 'occlusion', and 'blackout'.

#### 11.1.2 Adversarial Examples

Figure 2 exhibits several adversarial examples found by DeepConcolic (with $L_\infty$-norm and $L_0$-norm) and DeepXplore.

It is worth noting that, although DeepConcolic does not impose particular domain-specific constraints on the original image as DeepXplore does, the concolic testing procedure automatically generates test cases that resemble "human perception". For example, based on the $L_\infty$-norm, it produces adversarial examples (Figure 2, top row) that gradually reverse the black and white colours. For the $L_0$-norm, DeepConcolic generates adversarial examples similar to those of DeepXplore under the 'blackout' constraint, which is essentially pixel manipulation.

### 11.2 Concolic Testing Results on Different Test Criteria

This section presents the results of applying DeepConcolic to evaluate the test criteria NC, SSC, and NBC. DeepConcolic starts the NC testing with one single seed input. For SSC and NBC, to improve the performance, an initial set of 1000 images are sampled. Furthermore, for experimental purposes, we only test a subset of the neurons for SSC and NBC. A distance upper bound of 0.3 ($L_\infty$-norm) and 100 pixels ($L_0$-norm) is set up for collecting adversarial examples.

The full results are given in Table 3 and Figure 3. We have observed that the overhead for the symbolic analysis based on global optimization in Section 8.2 is too high. Thus, the SSC result with $L_0$-norm is excluded from the table.

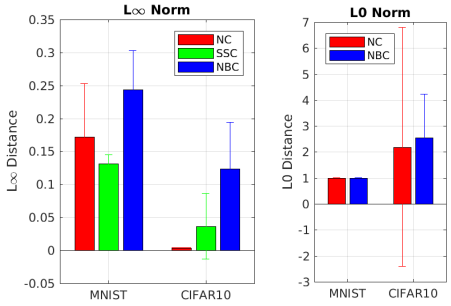
Overall, DeepConcolic achieves high coverage and detects a significant portion of adversarial examples, with the cover of corner-case activation values (i.e., NBC) sometimes being harder to achieve.

- Concolic testing is able to find adversarial examples with the minimum possible distance, that is, $\frac{1}{255} \approx 0.0039$ for the $L_\infty$ norm and 1 pixel for the $L_0$ norm. Figure 3 gives the average distance between adversarial examples, which often fall into a reasonably small distance range.

- The effectiveness of a criterion varies when the DNN under test changes. For example, subject to the $L_\infty$ norm, the NC seems to be more effective than SSC and NBC in the MNIST network with respect to the proportion

Table 3: Results for concolic testing using different test criteria, distance metrics and DNN models

| | $L_\infty$-norm | | | | | | $L_0$-norm | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | MNIST | | | CIFAR-10 | | | MNIST | | | CIFAR-10 | | |
| | coverage percentage | adversary /test suite | minimum distance | coverage percentage | adversary /test suite | minimum distance | coverage percentage | adversary /test suite | minimum distance | coverage percentage | adversary /test suite | minimum distance |
| **NC** | 97.89% | 9.69% | 0.0039 | 89.59% | 0.32% | 0.0039 | 97.24% | 0.07% | 1 | 99.69% | 13.91% | 1 |
| **SSC** | 94.10% | 0.37% | 0.1215 | 100% | 1.74% | 0.0039 | – | – | – | – | – | – |
| **NBC** | 60.74% | 0.83% | 0.0806 | 85.57% | 7.01% | 0.0113 | 48.52% | 0.06% | 1 | 100% | 4.30% | 1 |




(a) (b)

Figure 3: (a) Distance of NC, SSC, and NBC on MINIST and CIFAR-10 datasets based on $L_\infty$ norm; (b) Distance of NC and NBC on the two datasets based on $L_0$ norm.

of adversarial examples found; however, this is not the case when the CIFAR-10 network is tested. According to our results, different test criteria complement each other.

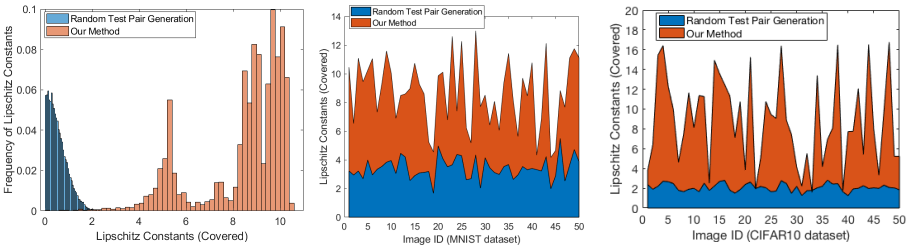
- Remarkably, for the same CIFAR-10 network, many more adversarial examples are found for the NC when the $L_0$-norm is used. This observation reflects the fact that, when designing test criteria for DNNs, they need to be examined using different distance metrics.

## 11.3 Experimental Results for Lipschitz Constant Testing

This section reports experimental results for Lipschitz constant testing on DNNs. We evaluate our concolic testing method on two DNNs trained on MNIST and CIFAR-10 with 99.4% and 76.6% testing accuracy, respectively. Their details are given in Appendix A.3.

### 11.3.1 Experimental Setup

We test Lipschitz constants ranging over $\{0.01 : 0.01 : 20\}$ on 50 MNIST images and $\{0.01 : 0.01 : 20\}$ on 50 CIFAR-10 images respectively chosen from testing datasets. Every image represents a subspace in $D_{L_1}$ and thus a requirement in Equation (11). We let $\Delta_{L_\infty} = 0.1$. The detailed parameter setup can be found in Appendix A.3.




(a) (b) (c)

Figure 4: (a) Lipschitz Constant Coverages generated by 1,000,000 randomly generated test pairs and our concolic testing method for input image-1 on MNIST DNN; (b) Lipschitz Constant Coverages generated by random testing and our method for 50 input images on MNIST DNN; (c) Lipschitz Constant Coverages generated by random testing and our method for 50 input images on CIFAR-10 DNN.

### 11.3.2 Baseline Method

Since this paper is the first to test Lipschitz constants of DNNs, as far as we know there is no existing work that can be directly used as a baseline. Thus, we compare our method with random test case generation. For this specific test requirement, given a predefined Lipschitz constant $c$, an input $t_0$ and the radius of norm ball (e.g., for $L_1$ and $L_2$ norms) or hypercube space (for $L_\infty$-norm) $\Delta$, we randomly generate two test pairs $t_1$ and $t_2$ that satisfy the space constraint (i.e., $||t_1 - t_0||_{D_2} \leq \Delta$ and $||t_2 - t_0||_{D_2} \leq \Delta$), and then check whether $\mathbf{Lip}(t_1, t_2) > c$ holds. We repeat the random generation until we find a satisfying test pair or the number of repetitions is larger than a predefined threshold. We set such threshold as $N_{rd} = 1,000,000$. Namely, if we randomly generate 1,000,000 test pairs and none of them can satisfy the Lipschitz constant requirement $> c$, we treat this test as a failure and return the largest Lipschitz constant found and the corresponding test pair; otherwise, we treat it as successful and return the satisfying test pair.

### 11.3.3 Experimental Results

Figure 4 (a) depicts the Lipschitz Constant Coverage generated by 1,000,000 random test pairs and our concolic test generation method for image-1 on MNIST DNNs. As we can see, even though we produce 1,000,000 test pairs by random test generation, the maximum Lipschitz converage reaches only 3.23 and most of the test pairs are in the range $[0.01, 2]$. Our concolic method, on the other hand, can cover a Lipschitz range of $[0.01, 10.38]$, where most cases lie in $[3.5, 10]$, which is poorly covered by random test generation.

Figure 4 (b) and (c) compare the Lipschitz constant coverage of test pairs from the random method and the concolic method on both MNIST and CIFAR-10 models. Our method significantly outperforms random test case generation. We note that covering a large Lipschitz constant range for DNNs is a challenging

problem since most image pairs (within a certain high-dimensional space) can produce small Lipschitz constants (such as 1 to 2). This explains the reason why randomly generated test pairs concentrate in a range of less than 3. However, for safety-critical applications such as self-driving cars, a DNN with a large Lipschitz constant essentially indicates it is more vulnerable to adversarial perturbations [13, 27]. As a result, a test method that can cover larger Lipschitz constants provides a useful robustness indicator for a trained DNN. We argue that, for safety testing of DNNs, the concolic test method for Lipschitz constant coverage can complement existing methods to achieve significantly better coverage.

# 12 Conclusions

In this paper, we propose the first concolic testing method for DNNs. We implement it in a software tool and apply the tool to evaluate DNN robustness, through coverage testing for Lipschitz continuity and several other test criteria. Experimental results confirm that the combination of concrete execution and symbolic analysis serves as a viable approach for DNN testing.

# References


[1] C. Kaner, "Exploratory testing," in *Quality Assurance Institute Worldwide Annual Software Testing Conference*, 2006.

[2] M. Wicker, X. Huang, and M. Kwiatkowska, "Feature-guided black-box safety testing of deep neural networks," in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems, arXiv preprint arXiv:1710.07859*, 2018. [Online]. Available: http://arxiv.org/abs/1710.07859

[3] K. Pei, Y. Cao, J. Yang, and S. Jana, "DeepXplore: Automated whitebox testing of deep learning systems," in *Proceedings of the 26th Symposium on Operating Systems Principles*. ACM, 2017, pp. 1–18.

[4] Y. Tian, K. Pei, S. Jana, and B. Ray, "DeepTest: Automated testing of deep-neural-network-driven autonomous cars," in *Proceedings of the 40th International Conference on Software Engineering, ICSE, Gothenburg, Sweden, May 27-June 3, 2018.*

[5] Y. Sun, X. Huang, and D. Kroening, "Testing deep neural networks," *ArXiv e-prints*, no. arXiv:1803.04792, 2018.

[6] L. Ma, F. Juefei-Xu, J. Sun, C. Chen, T. Su, F. Zhang, M. Xue, B. Li, L. Li, Y. Liu *et al.*, "DeepGauge: Comprehensive and multi-granularity testing criteria for gauging the robustness of deep learning systems," *arXiv preprint arXiv:1803.07519*, 2018.

[7] P. Godefroid, N. Klarlund, and K. Sen, "DART: directed automated random testing," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2005, pp. 213–223.

[8] K. Sen, D. Marinov, and G. Agha, "CUTE: a concolic unit testing engine for C," in *ACM SIGSOFT Software Engineering Notes*, vol. 30, no. 5. ACM, 2005, pp. 263–272.

[9] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *International Conference on Learning Representations (ICLR)*, 2014.

[10] T. Wiatowski and H. Bölcskei, "A mathematical theory of deep convolutional neural networks for feature extraction," *IEEE Transactions on Information Theory*, vol. 64, no. 3, pp. 1845–1866, 2018.

[11] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," *arXiv preprint arXiv:1701.07875*, 2017.

[12] R. Balan, M. Singh, and D. Zou, "Lipschitz properties for deep convolutional networks," *arXiv preprint arXiv:1701.05217*, 2017.

[13] W. Ruan, X. Huang, and M. Kwiatkowska, "Reachability analysis of deep neural networks with provable guarantees," *The 27th International Joint Conference on Artificial Intelligence (IJCAI)*, 2018.

[14] X. Huang, M. Kwiatkowska, S. Wang, and M. Wu, "Safety verification of deep neural networks," in *International Conference on Computer Aided Verification*. Springer, 2017, pp. 3–29.

[15] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient SMT solver for verifying deep neural networks," in *International Conference on Computer Aided Verification*. Springer, 2017, pp. 97–117.

[16] K. Hayhurst, D. Veerhusen, J. Chilenski, and L. Rierson, "A practical tutorial on modified condition/decision coverage," NASA, Tech. Rep., 2001.

[17] C. Cadar, D. Dunbar, D. R. Engler *et al.*, "KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs." in *OSDI*, vol. 8, 2008, pp. 209–224.

[18] T. Xie, D. Marinov, W. Schulte, and D. Notkin, "Symstra: A framework for generating object-oriented unit tests using symbolic execution," in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2005, pp. 365–381.

[19] W. Visser, C. S. Psreanu, and S. Khurshid, "Test input generation with Java PathFinder," *ACM SIGSOFT Software Engineering Notes*, vol. 29, no. 4, pp. 97–107, 2004.

[20] R. Kannavara, C. J. Havlicek, and B. Chen, "Challenges and opportunities with concolic testing," 2015.

[21] J. Burnim and K. Sen, "Heuristics for scalable dynamic test generation," in *Automated Software Engineering, ASE. 23rd International Conference on*. IEEE, 2008, pp. 443–446.

[22] P. Godefroid, M. Y. Levin, D. A. Molnar *et al.*, "Automated whitebox fuzz testing." in *NDSS*, vol. 8, 2008, pp. 151–166.

[23] X. Wang, J. Sun, Z. Chen, P. Zhang, J. Wang, and Y. Lin, "Towards optimal concolic testing," in *Proceedings of the 40th International Conference on Software Engineering, ICSE, Gothenburg, Sweden, May 27-June 3, 2018*.

[24] S. Cha, S. Hong, J. Lee, and H. Oh, "Automatically generating search heuristics for concolic testing," in *Proceedings of the 40th International Conference on Software Engineering, ICSE, Gothenburg, Sweden, May 27-June 3, 2018*.

[25] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.

[26] Z. Wang, E. P. Simoncelli, and A. C. Bovik, "Multiscale structural similarity for image quality assessment," in *Signals, Systems and Computers, Conference Record of the Thirty-Seventh Asilomar Conference on*, 2003.

[27] W. Ruan, M. Wu, Y. Sun, X. Huang, D. Kroening, and M. Kwiatkowska, "Global robustness evaluation of deep neural networks with provable guarantees for L0 norm," *arXiv preprint arXiv:1804.05805v1*, 2018.

[28] T. Roubicek, *Relaxation in Optimization Theory and Variational Calculus*. Berlin: Walter de Gruyter, 1997.

[29] C. Audet and W. Hare, *Derivative-Free and Blackbox Optimization*. Springer, 2017.

# A Appendix

## A.1 Settings for Comparison with DeepXplore

### A.1.1 Parameter Settings

- DeepConcolic
  - $L_\infty$-norm ball radius = 0.3
  - $L_0$-norm upper bound = 100
- DeepXplore
  - transformation = 'light', 'occl', 'blackout'
  - weight_diff = 1
  - weight_nc = 0.1
  - step = 10
  - seeds = 1
  - grad_iteration = 1000
  - threshold = 0
  - target_model = 0 (our model)
  - start_point = (14, 14)
  - occlusion_size = (10, 10)
  - constraint_black(rect_shape=(1, 1))

Note that, as DeepXplore needs more than one DNNs, in this case we set our trained DNN as the target model, and utilise the other two default models in DeepXplore[5].

### A.1.2 Platforms

- Hardware Platform:
  - Intel(R) Core(TM) i5-4690S CPU @ 3.20 GHz × 4
- Software Platform:
  - Fedora 26 (64-bit)
  - Anaconda, PyCharm

## A.2 Settings for Concolic Testing Results on Different Test Criteria

### A.2.1 Model Architecture

When evaluating the testing results of DeepConcolic, we train an MNIST DNN with architecture in Table 4, and a CIFAR-10 DNN with architecture in Table 5.

[5] http://github.com/peikexin9/deepxplore

Table 4: MNIST DNN architecture.

| Layer Type | MNIST |
|---|---|
| Convolution | $3 \times 3 \times 32$ |
| ReLU | |
| Convolution | $3 \times 3 \times 32$ |
| ReLU | |
| Max Pooling | $2 \times 2$ |
| Convolution | $3 \times 3 \times 64$ |
| ReLU | |
| Convolution | $3 \times 3 \times 64$ |
| ReLU | |
| Max Pooling | $2 \times 2$ |
| Flatten | |
| Fully Connected | 200 |
| ReLU | |
| Fully Connected | 200 |
| ReLU | |
| Fully Connected | 10 |
| Softmax | |

Table 5: CIFAR-10 DNN architecture.

| Layer Type | CIFAR-10 |
|---|---|
| Convolution | $3 \times 3 \times 32$ |
| ReLU | |
| Convolution | $3 \times 3 \times 32$ |
| ReLU | |
| Max Pooling | $2 \times 2$ |
| Convolution | $3 \times 3 \times 64$ |
| ReLU | |
| Convolution | $3 \times 3 \times 64$ |
| ReLU | |
| Max Pooling | $2 \times 2$ |
| Flatten | |
| Fully Connected | 512 |
| ReLU | |
| Fully Connected | 10 |
| Softmax | |

#### A.2.2 Platforms

- Hardware Platform:
  - NVIDIA GeForce GTX TITAN Black
- Software Platform:
  - Ubuntu 14.04.3 LTS
  - Anaconda

### A.3 Settings for Lispchitz Constant Coverage

#### A.3.1 Model Structures of MNIST DNN

See Table 6.

#### A.3.2 Model Structures of CIFAR-10 DNN

See Table 7.

#### A.3.3 DNN Training Setup

- Hardware: Notebook PC with I7-7700HQ, 16 GB RAM, GTX 1050 GPU
- Software: Matlab 2018a, Neural Network Toolbox, Image Processing Toolbox, Parallel Computing Toolbox, Optimization Toolbox

Table 6: MNIST DNN for Lipschitz Coverage

| Layer Type | Size |
|---|---|
| Input layer | $28 \times 28 \times 1$ |
| Convolution layer | $3 \times 3 \times 16$ |
| ReLU activation | |
| Convolution layer | $3 \times 3 \times 32$ |
| Batch-Normalization layer | |
| ReLU activation | |
| Convolution layer | $3 \times 3 \times 64$ |
| Batch-Normalization layer | |
| ReLU activation | |
| Convolution layer | $3 \times 3 \times 128$ |
| Batch-Normalization layer | |
| ReLU activation | |
| Dropout layer | 0.5 |
| Fully Connected layer | 256 |
| ReLU activation | |
| Dropout layer | 0.5 |
| Fully Connected layer | 10 |
| Softmax + Class output | |

Table 7: CIFAR-10 DNN for Lipschitz Coverage

| Layer Type | Size |
|---|---|
| Input layer | $32 \times 32 \times 3$ |
| Convolution layer | $3 \times 3 \times 32$ |
| ReLU activation | |
| Convolution layer | $3 \times 3 \times 32$ |
| ReLU activation | |
| Maxpooling layer | $2 \times 2$ |
| Dropout layer | 0.5 |
| Convolution layer | $3 \times 3 \times 64$ |
| ReLU activation | |
| Convolution layer | $3 \times 3 \times 64$ |
| ReLU activation | |
| Maxpooling layer | $2 \times 2$ |
| Dropout layer | 0.5 |
| Fully Connected layer | 512 |
| ReLU activation | |
| Fully Connected layer | 10 |
| Softmax + Class output | |

- Parameter optimization settings: SGDM, Max Epochs = 30, Mini-Batch Size = 128
- Training dataset: MNIST training dataset with 50,000 images; CIFAR-10 training dataset with 50,000 images
- Training accuracy: 100% for MNIST; 95.3% for CIFAR-10.
- Testing dataset: MNIST testing dataset with 10,000 images and CIFAR-10 testing dataset with 10,000 images
- Testing accuracy: 99.46% for MNIST and 76.6% for CIFAR-10

#### A.3.4 Algorithm Setup

- $\Delta = 0.1$
- The maximum number of iterations on each compass search is 150
- The maximum number of concolic executions is 30 for an input image
- Tested Images: Image-1 to Image-50 from the testing datasets of MNIST and CIFAR-10